Week 4: Web Application Architecture

Learning Objectives

By the end of this lecture, you should be able to:

- *Describe the layered (N-tier) architecture of a typical web application: presentation, application, and data tiers.*
- *Identify the components that sit around this core in production; reverse proxy, API gateway, cache, CDN, third-party services; and their individual security considerations.*
- *Explain the MVC pattern and where input validation and business logic should live within it.*
- *Apply the STRIDE threat-modeling framework to identify concrete threats at each architectural trust boundary.*
- *Explain Web Cache Poisoning, Web Cache Deception, and HTTP Request Smuggling as attacks that exploit the existence of intermediary components.*
- *Describe Zero Trust and mutual TLS (mTLS) for microservice-to-microservice traffic, and explain how SSRF exploits misplaced internal trust.*
- *Explain database-tier privilege separation: least privilege, and splitting read and write accounts.*

The three classic tiers

Since Week 1, our diagrams have shown three boxes: browser, server, database. That's actually a simplified version of the standard three-tier architecture: a presentation tier (what the user sees and interacts with), an application tier (the business logic that decides what to do with input), and a data tier (persistent storage). Splitting these into separate layers isn't just organizational, it's a security boundary. Code in the presentation tier should never talk to the database directly; every request has to pass through the application tier's logic first.

Production Architecture

Real applications add several components around this core, mostly for performance, reliability, and scale but each one changes the attack surface too.

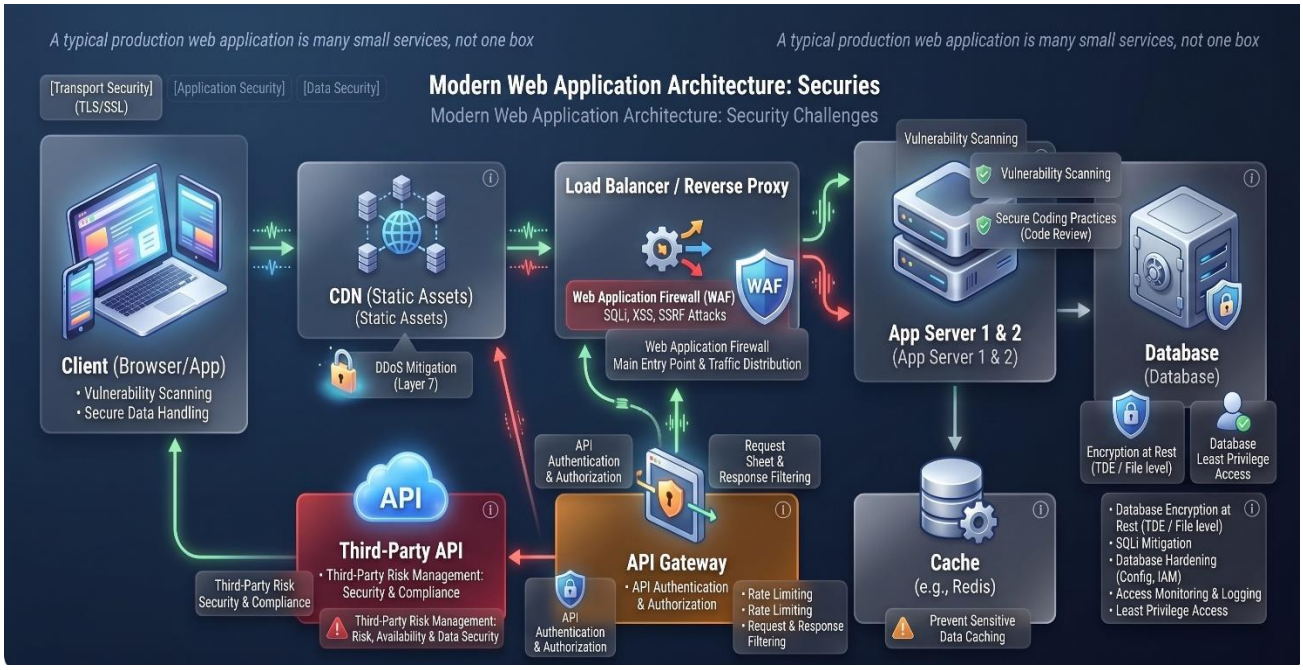


Figure 1 — A more realistic picture than "browser talks to server." Each box here is a separate piece of software with its own configuration to get right.

Component	Role	Key security consideration
Reverse Proxy / Load Balancer	Distributes traffic across servers, often terminates TLS	Must forward the real client IP and handle the Host header safely (Week 2's Host Header Injection).
API Gateway	Single entry point for backend/microservice APIs	Good place to centralize auth and rate limiting, but a single misconfiguration exposes everything behind it.
Application Server	Executes business logic	Where most injection and logic vulnerabilities actually live.
Cache (e.g. Redis)	Speeds up repeated reads	Can leak sensitive data across users if not access-controlled per session.
CDN	Serves static assets from edge locations	Cache-poisoning and cache-deception risk if misconfigured (see below).
Database	Persistent storage	Should never be directly reachable from the internet; use least-privilege accounts (see below).
Third-Party APIs	External integrations (payments, maps, analytics...)	Expands your attack surface beyond code you control; a supply-chain risk.

The MVC pattern

Within the application tier, most web frameworks organize code using Model-View-Controller (MVC). The Model holds data and business rules; the View renders what the user sees; the Controller receives incoming requests, decides what should happen, and coordinates the other two. From a security standpoint, this matters because it tells you where checks belong: input validation and authorization checks belong in the Controller and Model, before any business logic runs (never left to the View), and never assumed to have already happened somewhere upstream.

Trust boundaries

A trust boundary is any point where data crosses from a zone you don't control into a zone you do, or from a lower-privilege context into a higher-privilege one. The browser and the network between it and your server are always untrusted, no matter how the request looks. Once inside the application tier, data is still not automatically safe for the database; it has to cross one more boundary, ideally through parameterized queries.

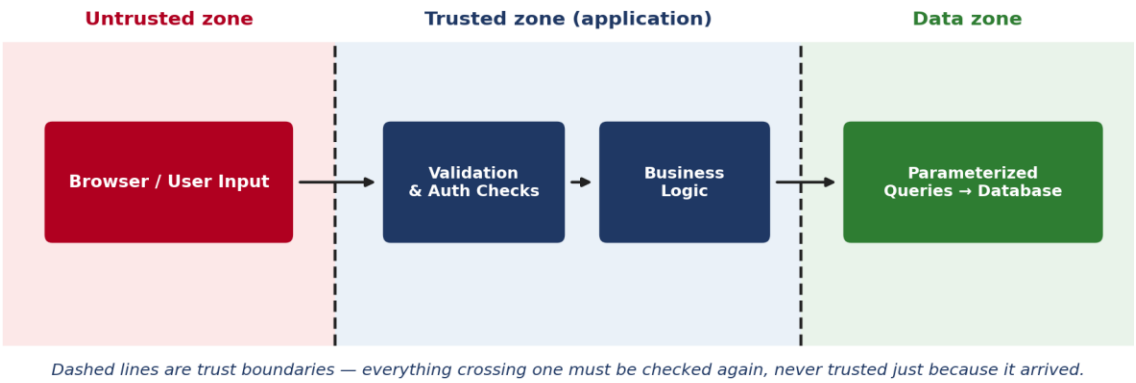


Figure 2 — The SQL Injection example from Week 1 is exactly what happens when the boundary before the database zone is skipped.

Making it concrete: applying STRIDE

"Draw boundaries and check them" is easy to say and vague to act on. STRIDE gives it structure: for each boundary, ask which of six threat categories applies: Spoofing (pretending to be someone/something else), Tampering (altering data in transit or at rest), Repudiation (denying an action with no way to prove otherwise), Information Disclosure (exposing data to those who shouldn't see it), Denial of Service (making the system unavailable), and Elevation of Privilege (gaining more access than granted). Applied to Figure 1's boundaries:

This is what "threat modeling" concretely means: walk the architecture diagram boundary by boundary, and for each one, ask which STRIDE category applies and what the defense is, before the system is built not after an incident.

Architectural boundary	Likely STRIDE threat	Impact & primary defense
Client → CDN / Proxy	Tampering, Denial of Service	Traffic altered or flooded in transit → TLS everywhere, rate limiting, a WAF.
Proxy → App Server	Spoofing, Information Disclosure	Forged client IP or Host header (Week 2) → validate X-Forwarded-For, allow-list real domains.
App Server → Database	Tampering, Elevation of Privilege	SQL Injection (Week 1) or an over-privileged DB account → parameterized queries + least-privilege accounts.
App Server → Other Microservices	Spoofing, Elevation of Privilege	Internal network wrongly assumed "safe" → mutual TLS + Zero Trust (see below).

When the intermediary itself is exploited: cache attacks

A CDN or cache sits between the user and your origin server specifically to avoid re-computing the same response repeatedly. That convenience creates two distinct attack classes. Web Cache Poisoning abuses "unkeyed inputs", headers or parameters that influence the response but are not part of what the cache uses to identify it, such as X-Forwarded-Host. If the origin reflects that header into the page and the cache key ignores it, one malicious request poisons the cached copy served to everyone else afterward.

Web Cache Deception works the opposite way: instead of poisoning shared content, it tricks the cache into storing a private one. A request like /account/profile.css can hit a dynamic, authenticated endpoint that some servers still serve normally regardless of the fake static-looking extension but the cache, fooled by the *.css* suffix, stores that personal page as if it were a static asset, making one user's private data retrievable by anyone who requests the same URL.

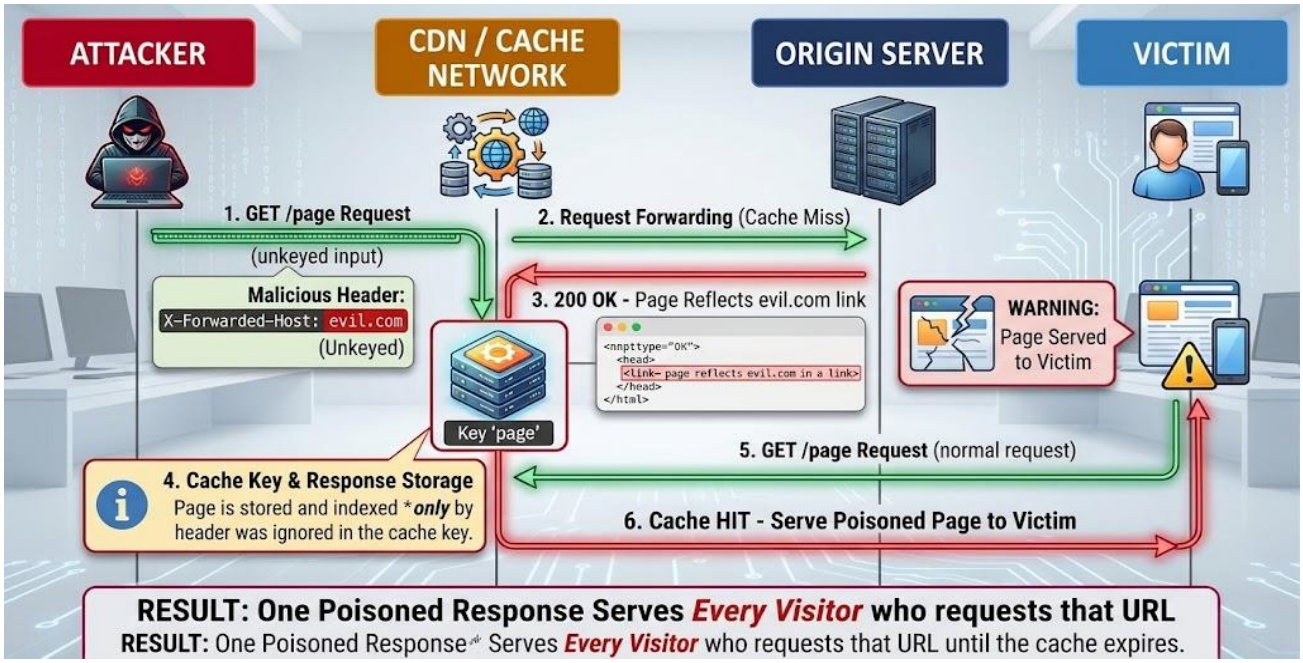


Figure 3 — The attacker only needs to hit the cache once; every later visitor to that same URL inherits the poisoned response.

HTTP Request Smuggling: when the proxy and backend disagree

Week 2 showed that a message body's length can be declared two different ways: Content-Length (a byte count) or Transfer-Encoding: chunked (a series of self-describing chunks). Request smuggling happens when a request includes both headers and the reverse proxy trusts one while the backend trusts the other, most classically "CL.TE", where the proxy uses Content-Length and the backend uses Transfer-Encoding.

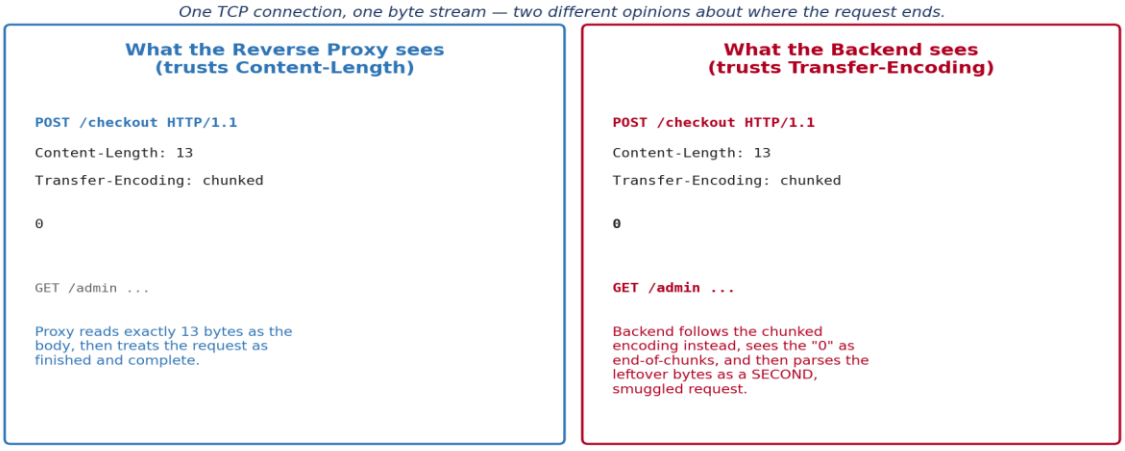


Figure 4 — The proxy forwards what it thinks is one complete, harmless request. The backend, parsing the same bytes differently, finds a second request hidden inside — one the proxy's security rules never inspected.

The danger is that any security filtering the proxy does (blocking /admin, for instance) only ever sees the outer request; the smuggled inner request reaches the backend having skipped that filter entirely. The fix is architectural, not a single patch: keep proxy and backend HTTP parsing strictly consistent, and prefer HTTP/2 end-to-end where the framing itself removes this ambiguity.

Monolith vs. microservices

A monolith runs the entire application as one deployable unit simpler to build and reason about, but a single vulnerability can potentially expose everything, sharing one blast radius. A microservices architecture splits the application into many small, independently deployed services that talk to each other over the network, limiting how far a single compromised service can reach, but multiplying the number of network boundaries that each need their own authentication and validation, exactly like the fourth row of the STRIDE table above.

Securing the seams: Zero Trust and mTLS

A common and dangerous assumption in microservices is that traffic from "inside the network" (localhost, the private VPC) can be trusted by default. Zero Trust Network Access (ZTNA) rejects that assumption outright: no request is trusted just because of where it came from — every service-to-service call is authenticated and authorized, every time. The standard mechanism is mutual TLS (mTLS): unlike ordinary HTTPS, where only the server proves its identity to the browser, mTLS has both sides present certificates, so Service A cryptographically proves who it is to Service B and vice versa. In larger deployments this is often handled by a service mesh, a dedicated infrastructure layer that automatically wraps every internal call in mTLS without each service having to implement it individually.

Skipping this is exactly how Server-Side Request Forgery (SSRF, OWASP A10:2021) becomes dangerous in a microservices world: if the application server will fetch any URL it's given, and internal services trust anything arriving from "inside," an attacker who tricks the app server into requesting an internal-only address can reach services that were never meant to face the internet — including, on cloud platforms, internal metadata endpoints (such as 169.254.169.254) that can hand over cloud credentials outright.

Database-tier security

The database deserves the same least-privilege thinking as everything else. The application should never connect as a superuser: separate accounts by function, so a bug or an injection in a read-only endpoint can't be leveraged to write or delete data. Connection pooling (reusing a small set of open database connections instead of

opening a new one per request) should be configured per account type as well, so a compromised read path can't silently borrow write-level credentials.

Account	Privileges	Used by
App read account	SELECT only	Reporting pages, search, read-heavy endpoints
App write account	SELECT, INSERT, UPDATE (no DROP/ALTER)	Core transactional logic (checkout, profile updates)
Migration / admin account	Full schema privileges	CI/CD migration jobs only — never the running application

Insecure design and the OWASP Top 10

This week's theme maps to OWASP A04:2021 – Insecure Design, a category about flaws baked into the architecture itself — before a single line of vulnerable code is even written. STRIDE, applied above, is exactly the kind of threat modeling this category asks for: during design, deliberately ask "what could go wrong here, and for whom?" for every trust boundary and every component, rather than only reacting to vulnerabilities discovered after the system is built. A well-coded implementation of a badly designed architecture is still insecure.

Putting it together: a booking/payment application

Consider a realistic booking-and-payment app built on Figure 1's shape. The Redis cache holds session data and should sit on a private network segment, reachable only from the App Server tier — never exposed to the CDN, the browser, or the public internet directly. Payment-provider API keys and database credentials live in a secrets manager, injected into the App Server at startup, never hard-coded and never returned in any API response. The App Server itself should be explicitly forbidden from making outbound requests to internal IP ranges or cloud metadata addresses unless a specific, allow-listed feature requires it — the concrete control that prevents the SSRF scenario described above. And when the App Server calls the payment microservice internally, that call goes over mTLS, authenticated like any external request, not waved through because it happens to originate "inside." Every one of these decisions is a direct application of a rule introduced earlier in this lecture — least privilege, Zero Trust, and a trust boundary that actually gets checked.